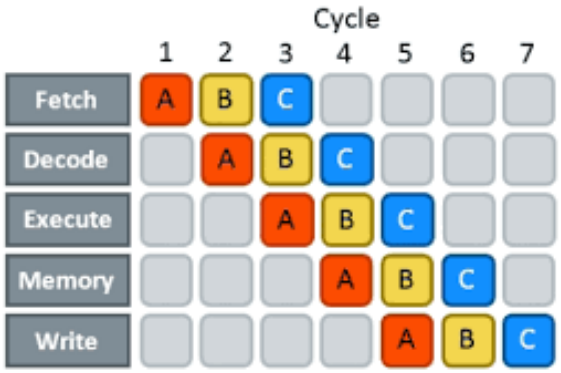
Thinking in parallel

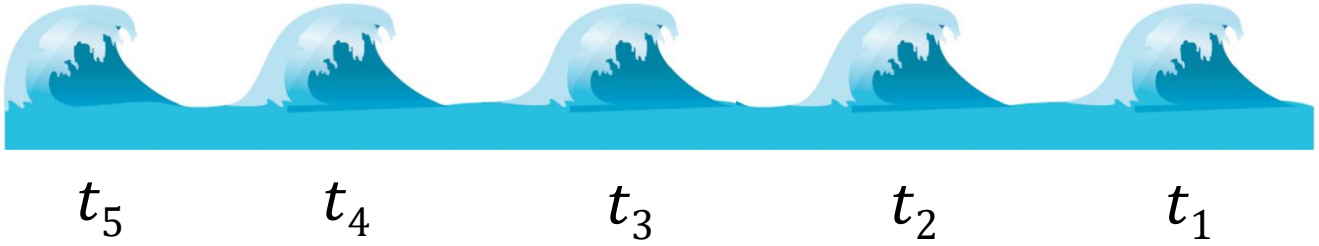
How to design algorithms for multi-processor systems

Basic techniques

Basic techniques: Pipelining

- Useful when we can execute several different operations concurrently
  - suitable for several data structures
- How it works
  - Break the task t into a sequence of subtasks $t_1, t_2, \ldots, t_k$, where t_i depends only on some of the operations that should be done by t_{i-1}
  - When all the operations of t_{i-1} that blocked t_i from being executed have been done (t_{i-1} may still be running), we can start executing t_i concurrently



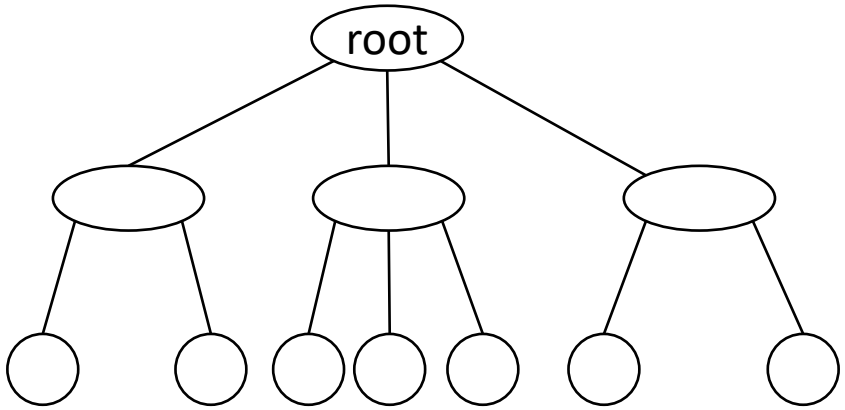


(2,3)-trees

DEFINITION: A *(2,3)-tree* is a rooted tree in which each internal node has either 2 or 3 children, and every path from the root to a leaf is of the same length.

LEMMA: The height of a (2,3)-tree with n leaves is $\Theta(\log n)$.

SKETCH OF PROOF: The smallest (2,3)-tree of height h contains 2^h leaves (every internal node has 2 children). The largest (2,3)-tree of height h contains 3^h leaves (every internal node has 3 children).

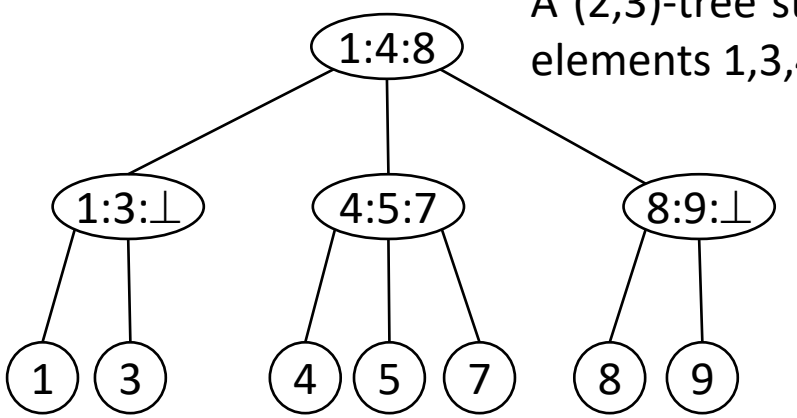


(2,3)-tree data structure

- A *(2,3)-tree* is also a data structure used to store n elements $a_1, a_2, \dots, a_n$ in a sorted order in the leaves of the tree. Each internal node v stores a triple $L[v]: M[v]: R[v]$ such that
  - $L[v]$: minimum element stored in the subtree rooted at its first (leftmost) child
  - $M[v]$: minimum element stored in the subtree rooted at its second (middle) child
  - $R[v]$: minimum element stored in the subtree rooted at its third (rightmost) child, if it exists
- Operations supported:
  - Search element ($O(\log n)$ time sequential)
  - Insert element ($O(\log n)$ time sequential)
  - Delete element ($O(\log n)$ time sequential)

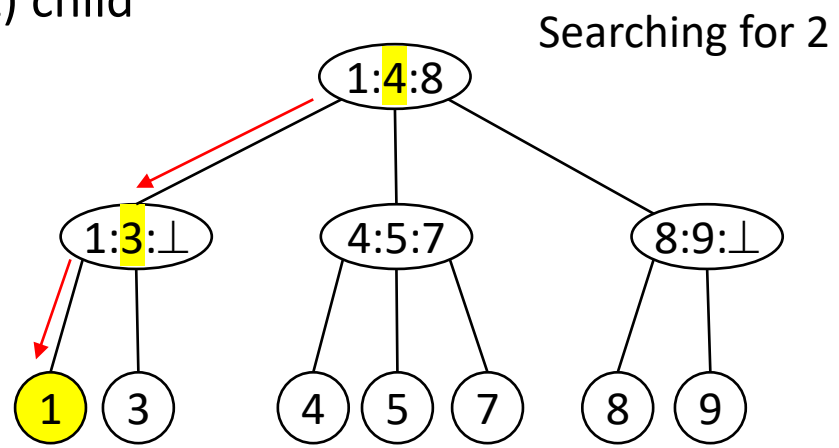
ASSUMPTION: W.l.o.g., all the elements stored are distinct

A (2,3)-tree storing the elements 1,3,4,5,7,8,9



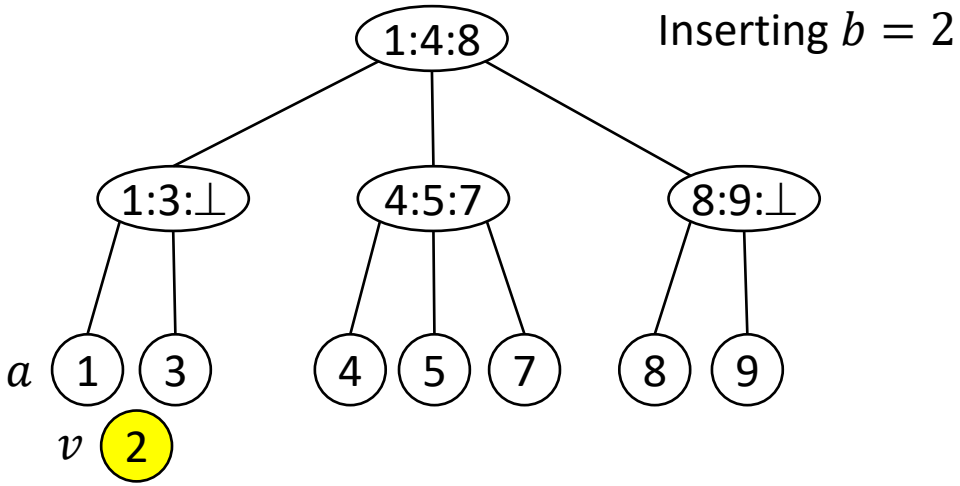
(2,3)-tree data structure: searching for an element

- **PROBLEM:** Given an element b, find the max element a such that $a \leq b$
- **SOLUTION:** Binary search approach
  - Start from the root v
  - While visiting an internal node v with $L[v]$: $M[v]$: $R[v]$
    - $b < M[v]$: move to the first (leftmost) child
    - $M[v] \leq b < R[v]$: move to the second (middle) child
    - $R[v] \leq b$: move to the third (rightmost) child
  - The predecessor of b is the reached leaf
    - with some exceptions (e.g., b is the minimum)



(2,3)-tree data structure: inserting an element

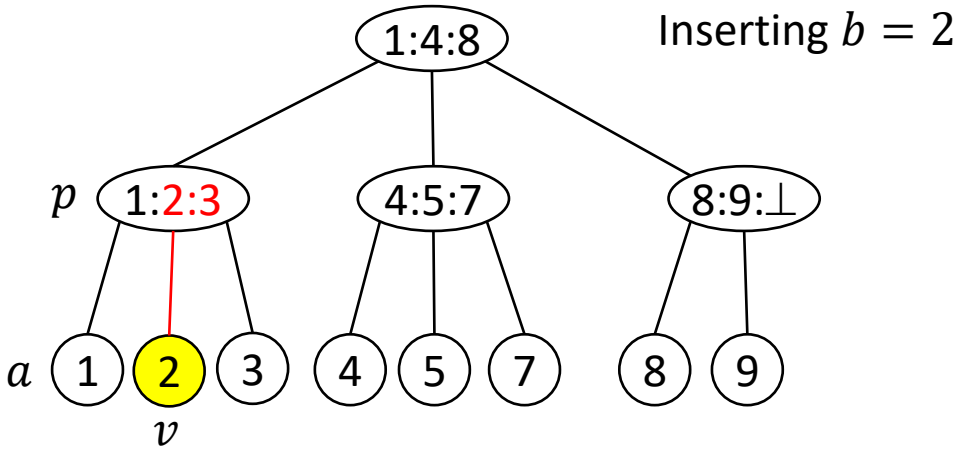
- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b



* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a

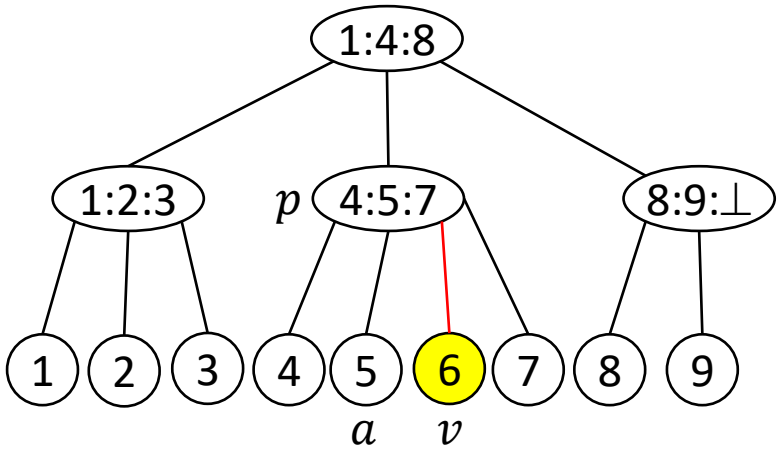


* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a

Inserting $b = 6$

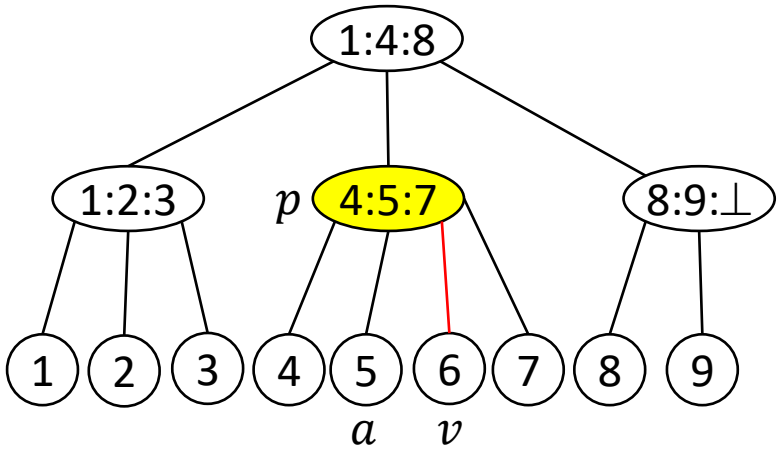


* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a
  - As long as p has four children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)

Inserting $b = 6$

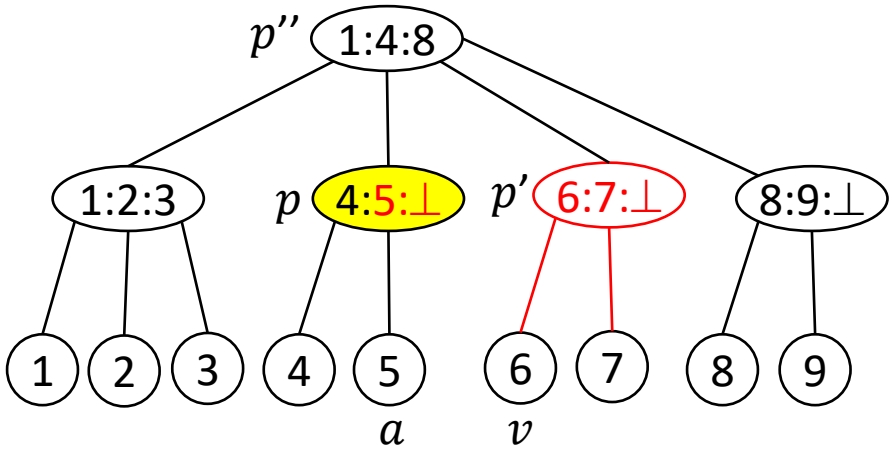


* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a
  - As long as p has four children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)

Inserting $b = 6$

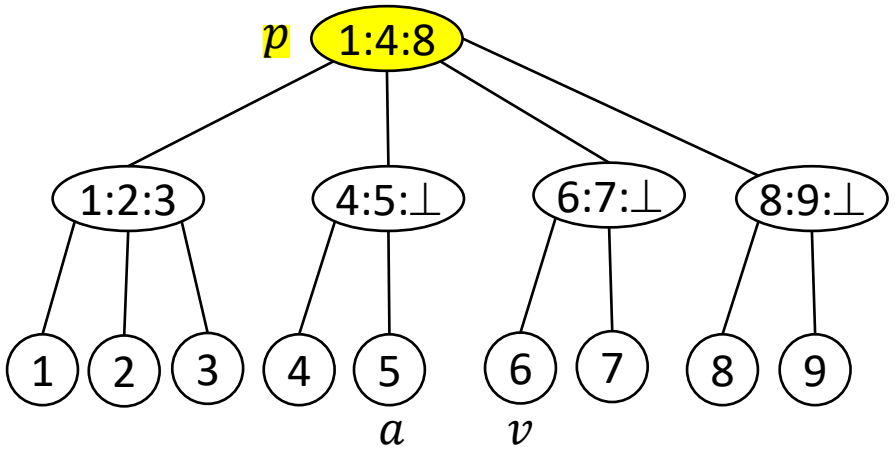


* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a
  - As long as p has four children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''

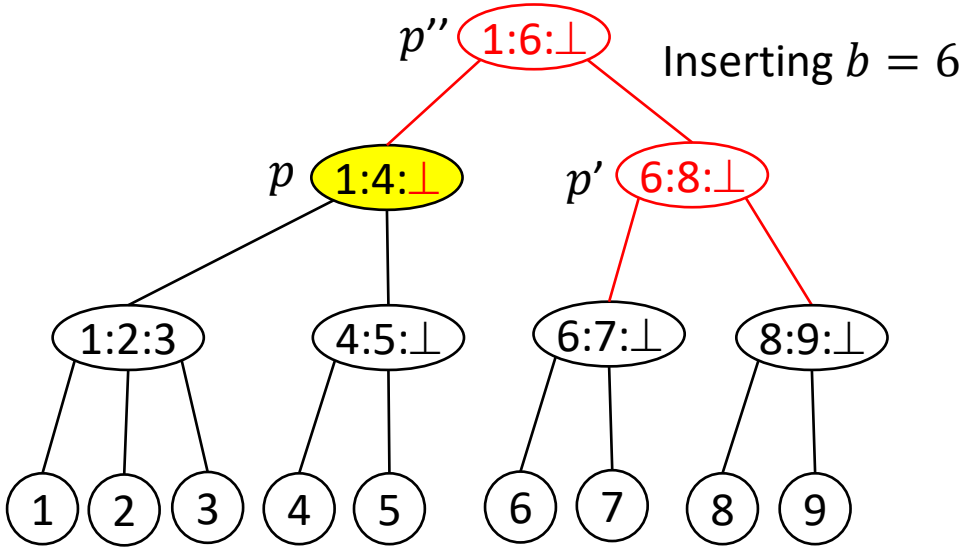
Inserting $b = 6$



* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

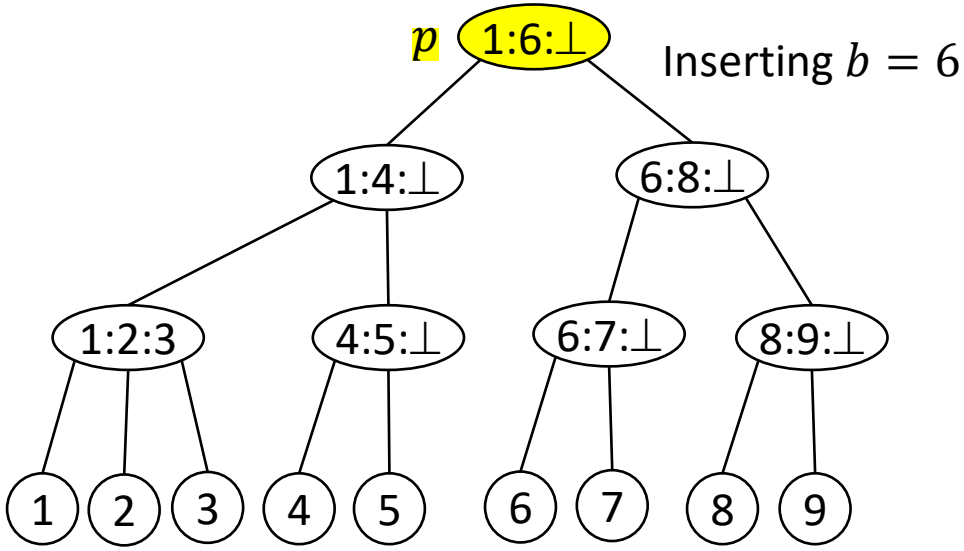
- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a
  - As long as p has four children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p'' (if p'' does not exist, then create it)
    - Update p with p''



* We do not discuss in depth how to update the auxiliary info in the internal nodes

(2,3)-tree data structure: inserting an element

- **PROBLEM:** Given an element b, insert it in the (2,3)-tree
- **SOLUTION:** Search for b and obtain the max element a such that $a \leq b$. If $a = b$ do nothing. Otherwise
  - create a leaf v holding the element b
  - insert v to the immediate right of a, with the same parent, say p, of a
  - As long as p has four children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''



* We do not discuss in depth how to update the auxiliary info in the internal nodes

Inserting k elements in a (2,3)-tree

INSERT ELEMENTS IN A (2,3)-TREE
Input: A (2,3)-tree of n elements $a_1 < a_2 < \cdots < a_n$ and $k = O(n)$ new elements $b_1 < b_2 < \cdots < b_k$ to be inserted in the tree.
Output: The updated (2,3)-tree.

- Easy $O(k \log n)$ time sequential algorithm
  - Insert the elements one after the other

$\Rightarrow$ EREW PRAM algorithm with time $O(k \log n)$ and work $O(k \log n)$ (work optimal)

- **GOAL:** PRAM algorithm with time $O(\log n)$ and work $O(k \log n)$ (work optimal)

Inserting k elements in a (2,3)-tree

INSERT ELEMENTS IN A (2,3)-TREE
Input: A (2,3)-tree of n elements $a_1 < a_2 < \cdots < a_n$ and $k = O(n)$ new elements $b_1 < b_2 < \cdots < b_k$ to be inserted in the tree.
Output: The updated (2,3)-tree.

- Simplifying assumption:
  - Insert b_1 (i.e., min) and b_k (i.e., max) in $O(\log n)$ time using the sequential algorithm
  - This guarantees that every leftover b_i has a predecessor a_j and a successor a_{j+1} in the tree
  - It takes time $O(\log n)$ and work $O(\log n)$

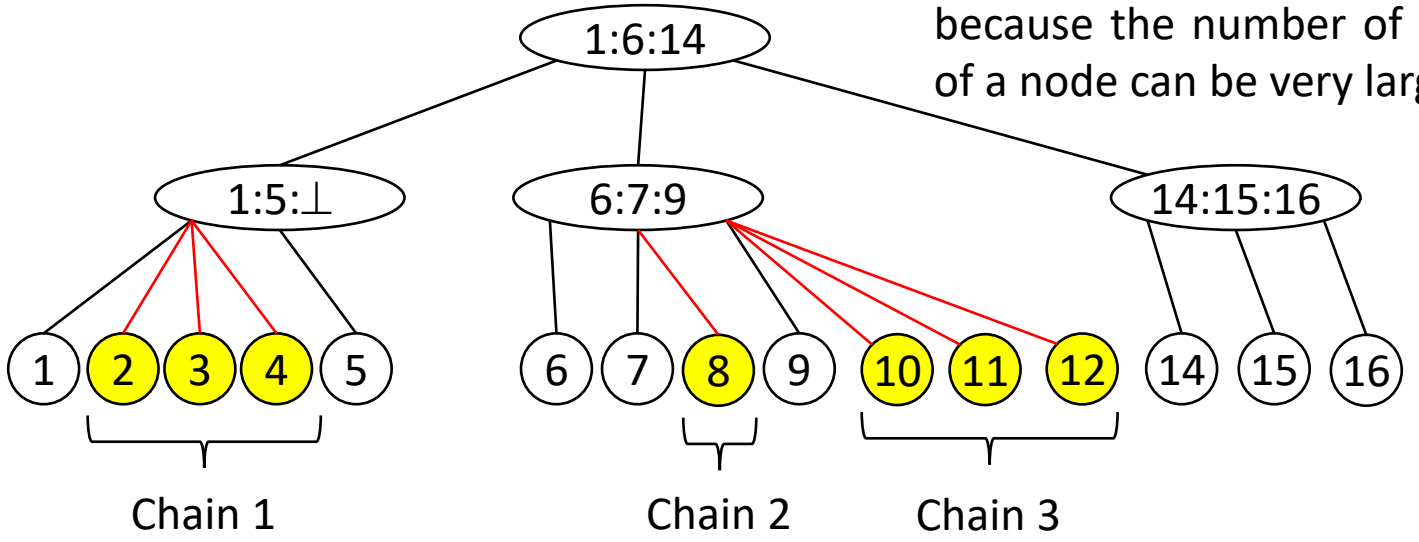
Inserting k elements in a (2,3)-tree

- Concurrently search for all elements b_i using the search procedure on each b_i separately
  - Time $O(\log n)$ and work $O(k \log n)$ on a CREW PRAM
    - (we can traverse the same internal node while searching for the new elements concurrently)
- A chain is a maximal sequence of b_i elements that must be placed between two consecutive leaves of the tree

ISSUE: Difficult to parallelize because the number of children of a node can be very large

Insert elements

2 3 4 8 10 11 12

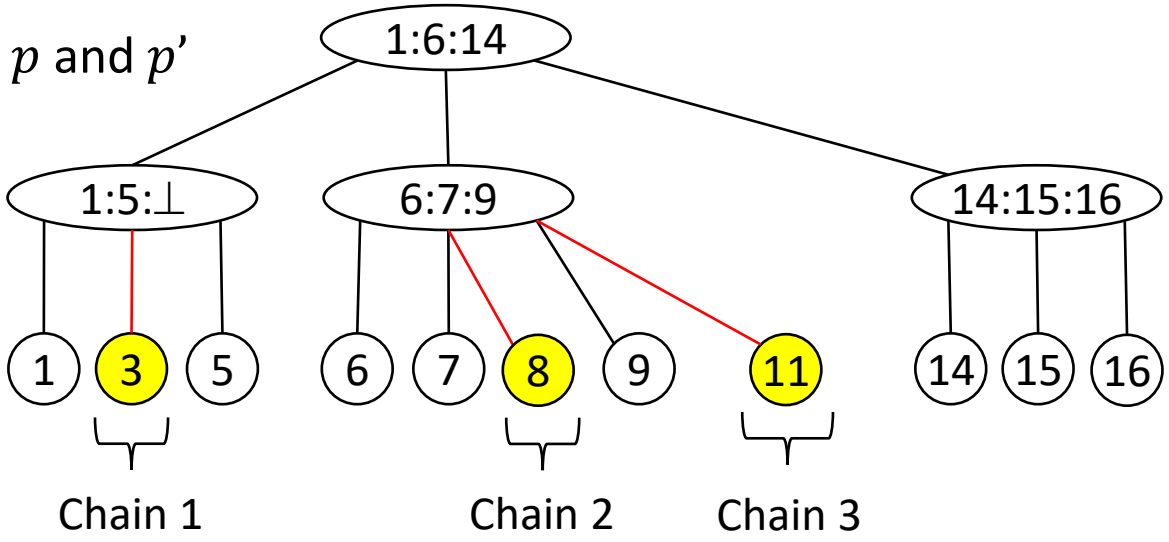


Inserting k elements in a (2,3)-tree: a special case

- Consider a special case in which each chain contains exactly one element
  - Easier to parallelize as each node has <u>at most 6 children</u>
- Insert the elements in parallel
  - insert v to the immediate right of its predecessor, say a, with the same parent, say p, of a
  - As long as p has four <u>or more</u> children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''

Insert elements

3 8 11

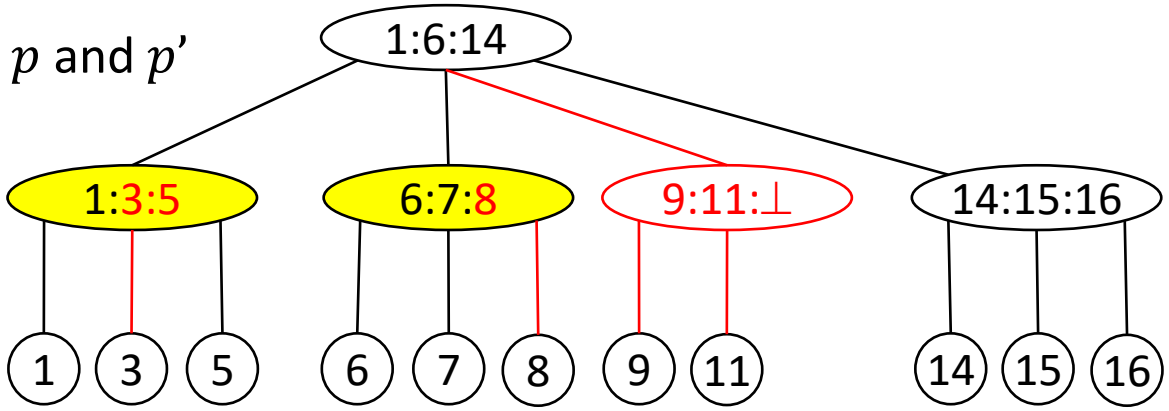


Inserting k elements in a (2,3)-tree: a special case

- Consider a special case in which each chain contains exactly one element
  - Easier to parallelize as each node has <u>at most 6 children</u>
- Insert the elements in parallel
  - insert v to the immediate right of its predecessor, say a, with the same parent, say p, of a
  - As long as p has four <u>or more</u> children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''

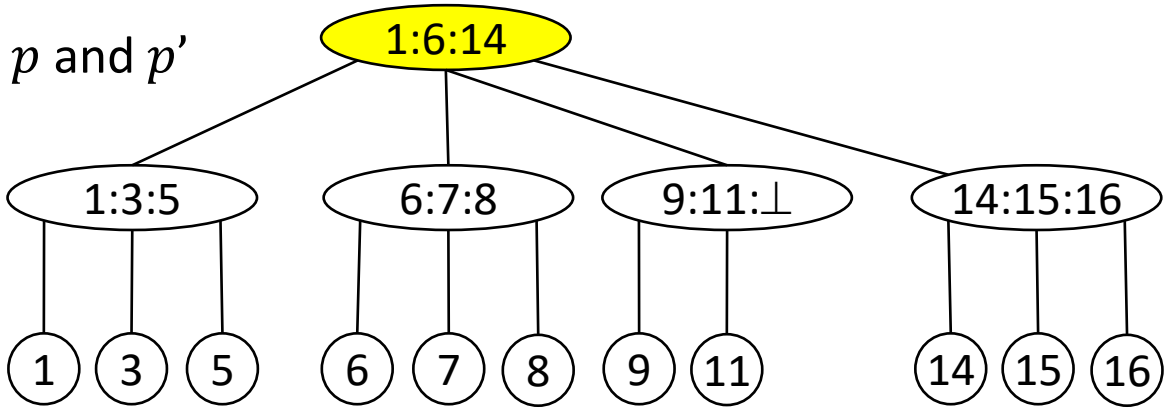
Insert elements

3 8 11



Inserting k elements in a (2,3)-tree: a special case

- Consider a special case in which each chain contains exactly one element
  - Easier to parallelize as each node has <u>at most 6 children</u>
- Insert the elements in parallel
  - insert v to the immediate right of its predecessor, say a, with the same parent, say p, of a
  - As long as p has four <u>or more</u> children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''

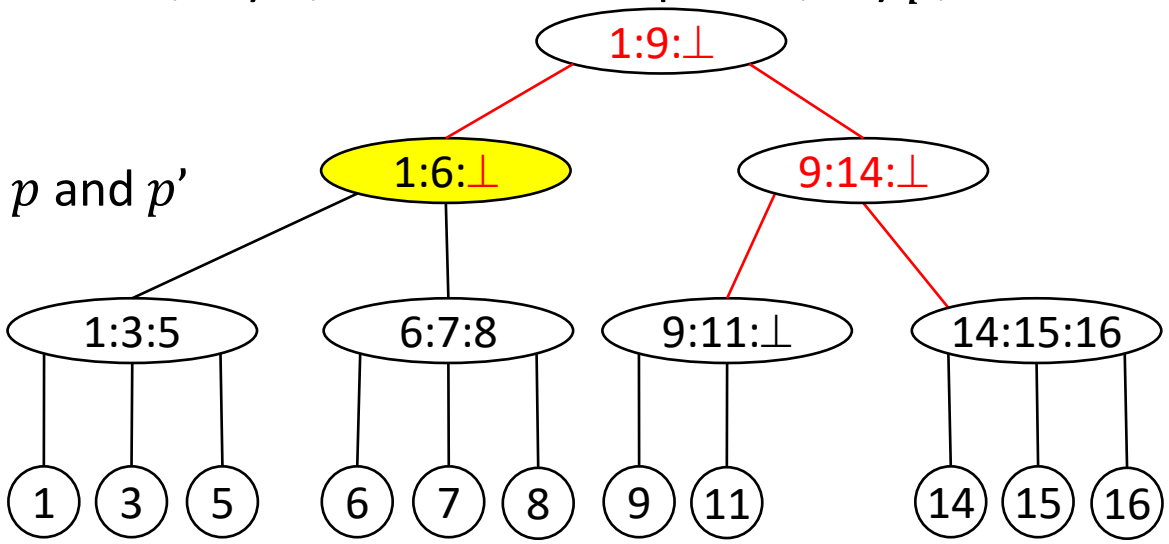


Insert elements

3 8 11

Inserting k elements in a (2,3)-tree: a special case

- Consider a special case in which each chain contains exactly one element
  - Easier to parallelize as each node has <u>at most 6 children</u>
- Insert the elements in parallel
  - insert v to the immediate right of its predecessor, say a, with the same parent, say p, of a
  - As long as p has four <u>or more</u> children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''



Insert elements

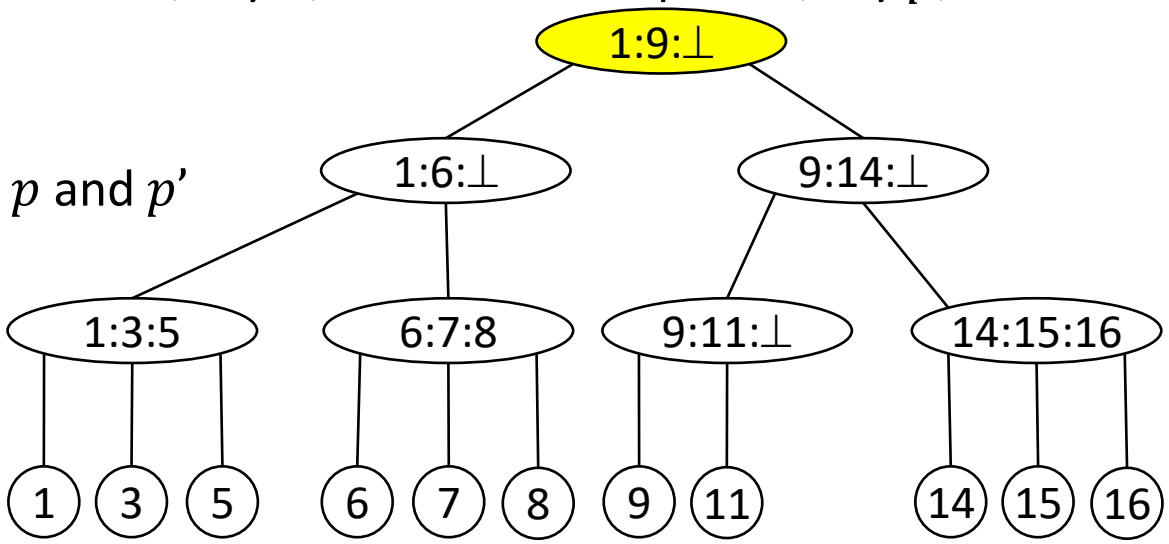
3 8 11

Inserting k elements in a (2,3)-tree: a special case

- Consider a special case in which each chain contains exactly one element
  - Easier to parallelize as each node has <u>at most 6 children</u>
- Insert the elements in parallel
  - insert v to the immediate right of its predecessor, say a, with the same parent, say p, of a
  - As long as p has four <u>or more</u> children
    - create a new node p'
    - distribute the children of p evenly to p and p'
    - Attach p' to the parent of p, say p''
      (if p'' does not exist, then create it)
    - Update p with p''

Insert elements

3 8 11



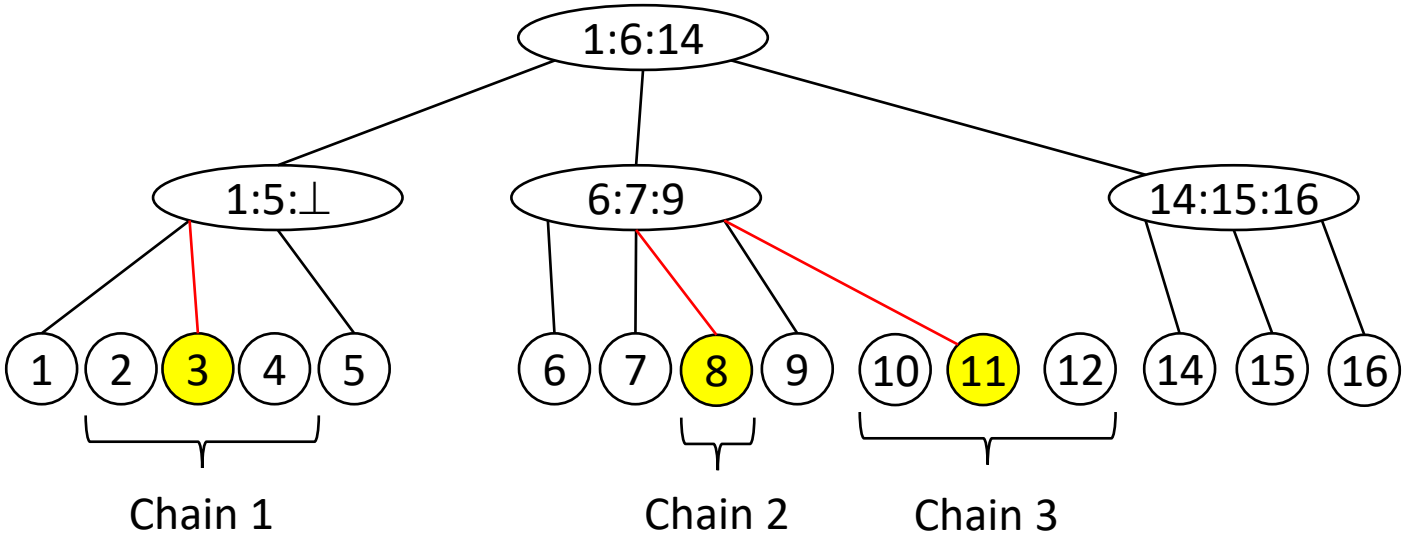
Time $O(\log n)$ and work $O(k \log n)$ on a CREW PRAM

Inserting k elements in a (2,3)-tree

- Use Divide and Conquer to reduce the general case to the special case
  - insert only the median element of each chain, for all chains, concurrently

Insert elements

2 3 4 8 10 11 12

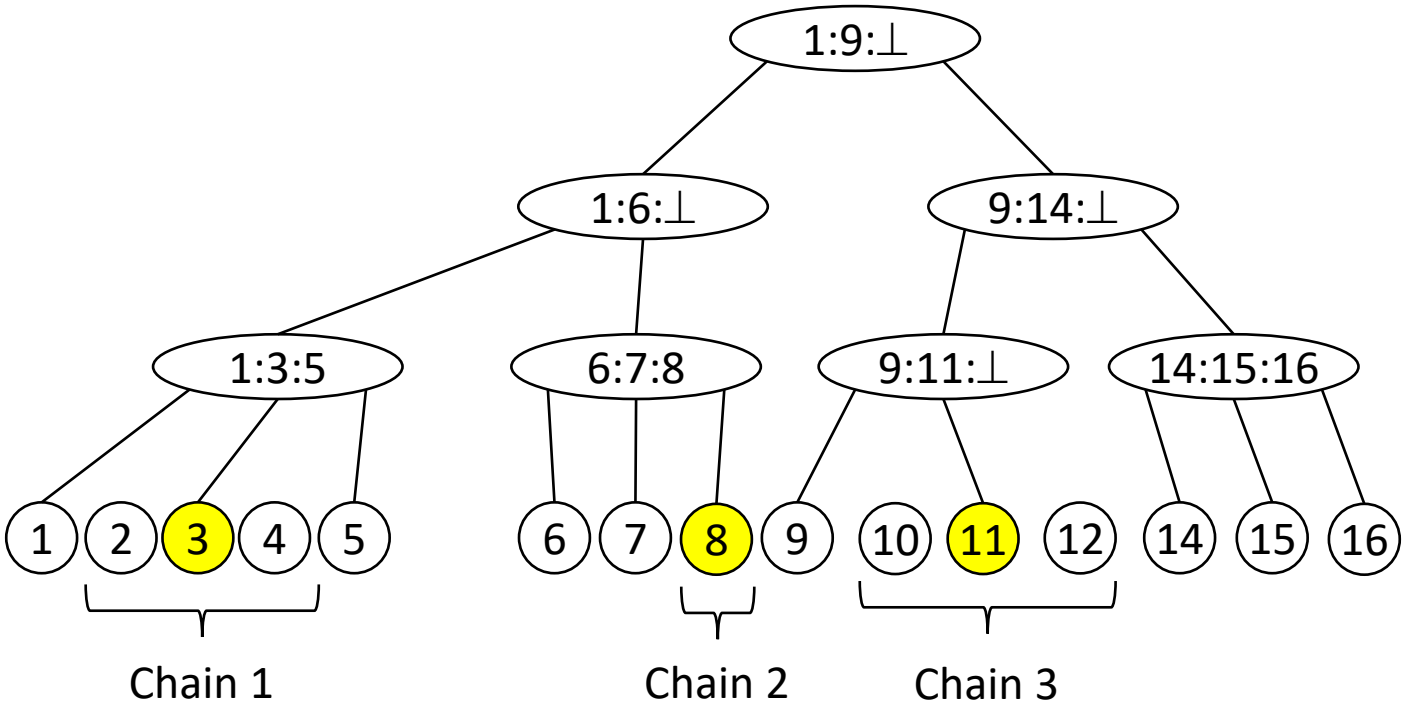


Inserting k elements in a (2,3)-tree

- Use Divide and Conquer to reduce the general case to the special case
  - insert only the median element of each chain, for all chains, concurrently

Insert elements

2 3 4 8 10 11 12

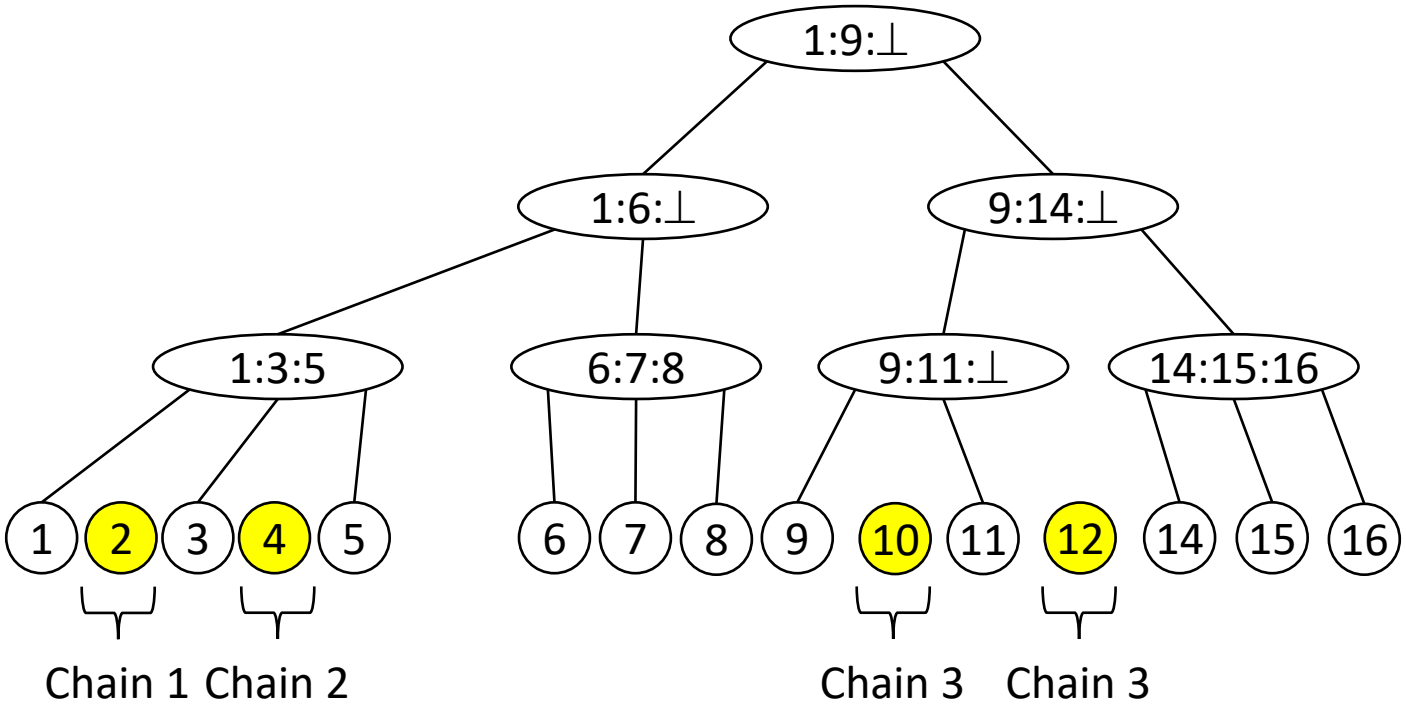


Inserting k elements in a (2,3)-tree

- Use Divide and Conquer to reduce the general case to the special case
  - insert only the median element of each chain, for all chains, concurrently
  - the new chains have a halved size
  - recurse

Insert elements

2 3 4 8 10 11 12

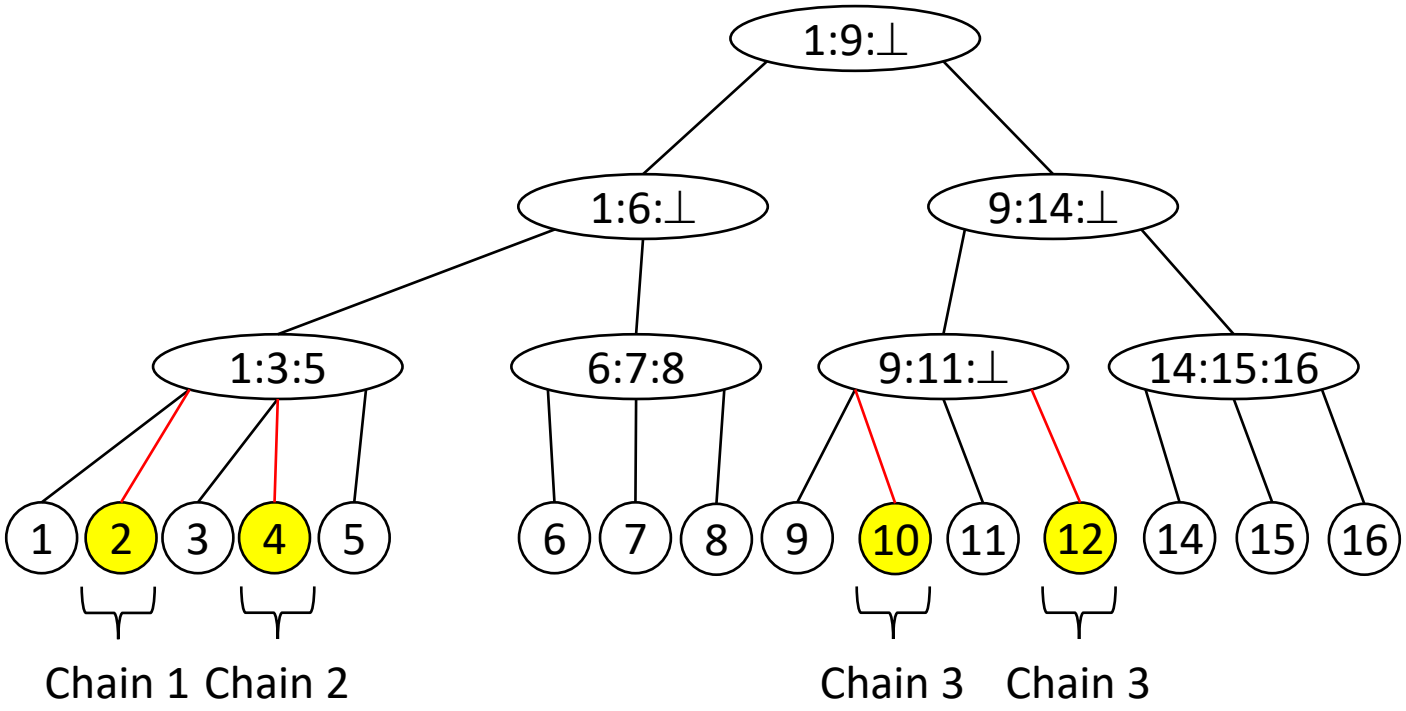


Inserting k elements in a (2,3)-tree

- Use Divide and Conquer to reduce the general case to the special case
  - insert only the median element of each chain, for all chains, concurrently
  - the new chains have a halved size
  - recurse

Insert elements

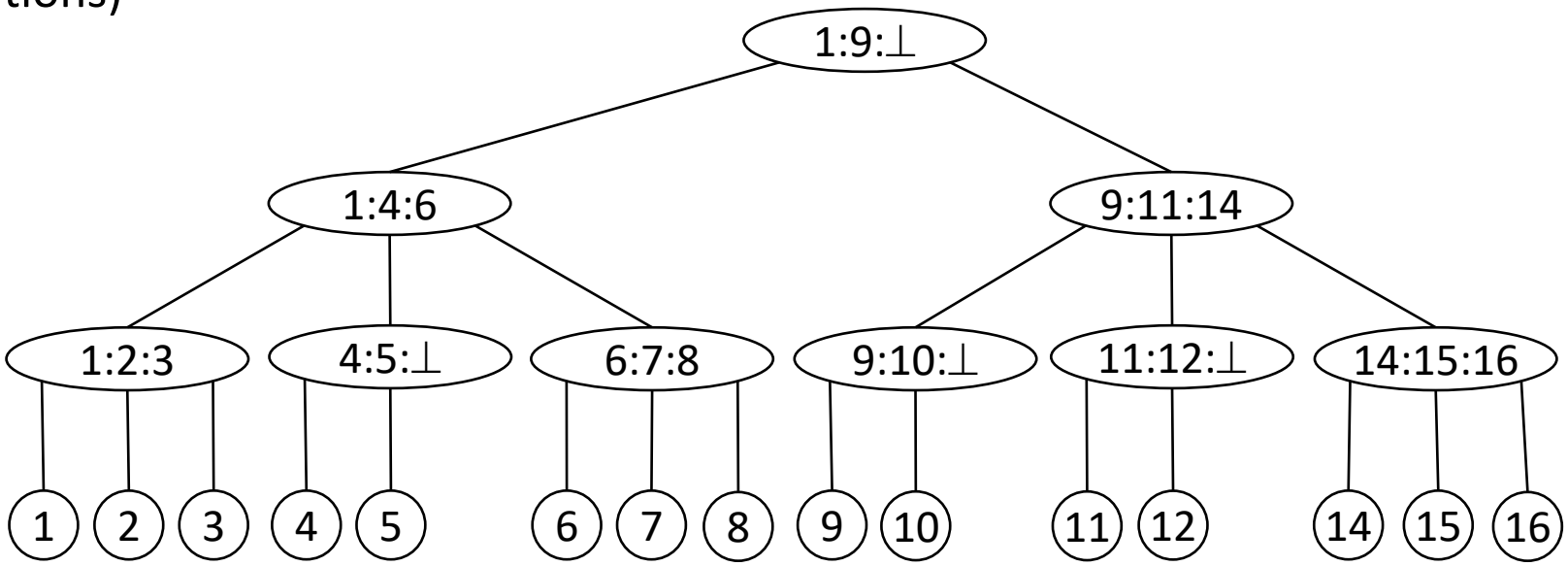
2 3 4 8 10 11 12



Inserting k elements in a (2,3)-tree

- Use Divide and Conquer to reduce the general case to the special case
  - insert only the median element of each chain, for all chains, concurrently
  - the new chains have a halved size
  - recurse ($O(\log k)$ iterations)

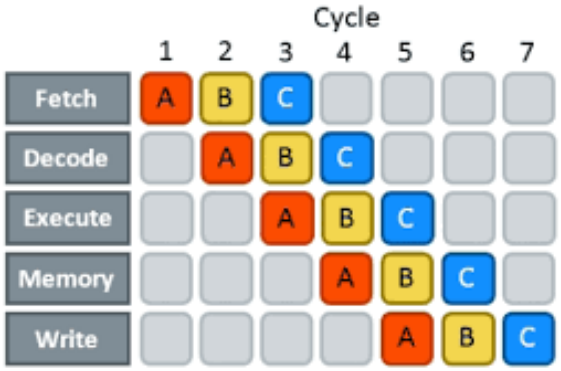
Insert elements



Time $O(\log k \cdot \log n)$ and work $O(k \log n)$ on a CREW PRAM

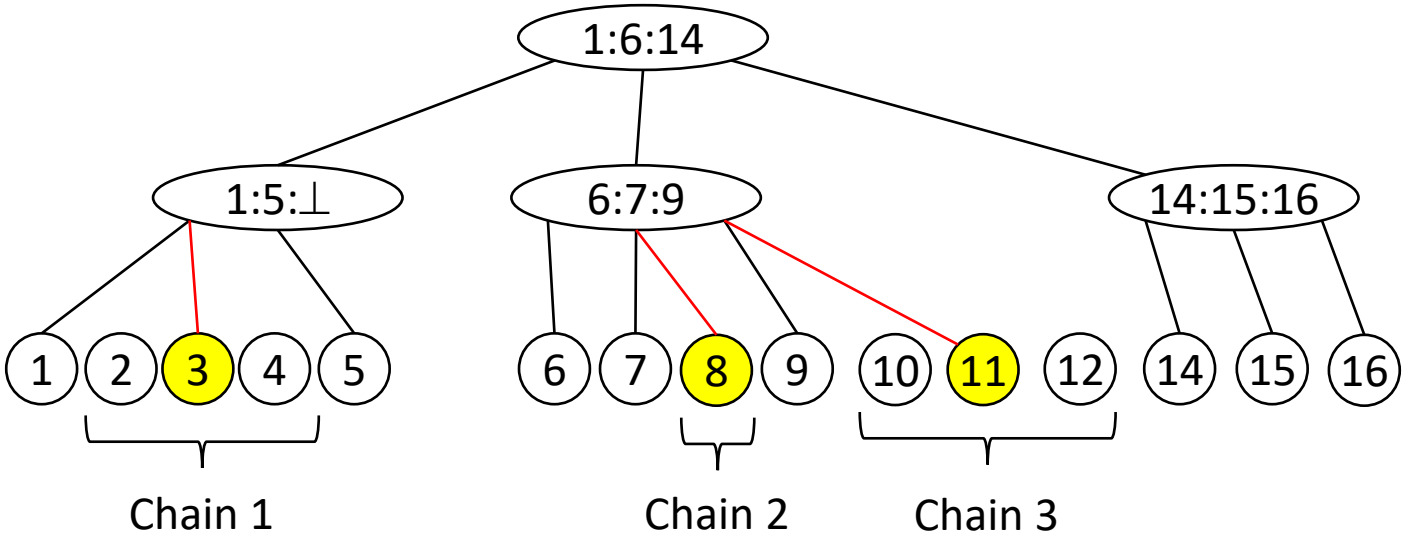
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



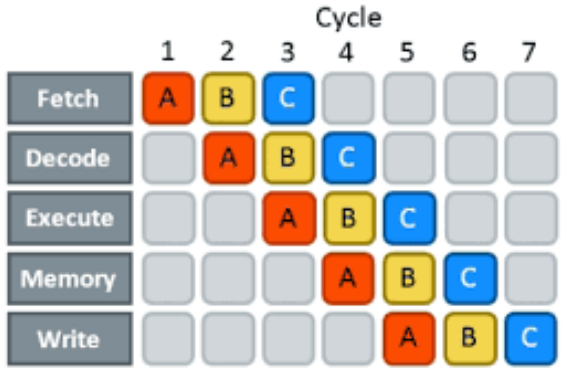
Insert elements

2 3 4 8 10 11 12



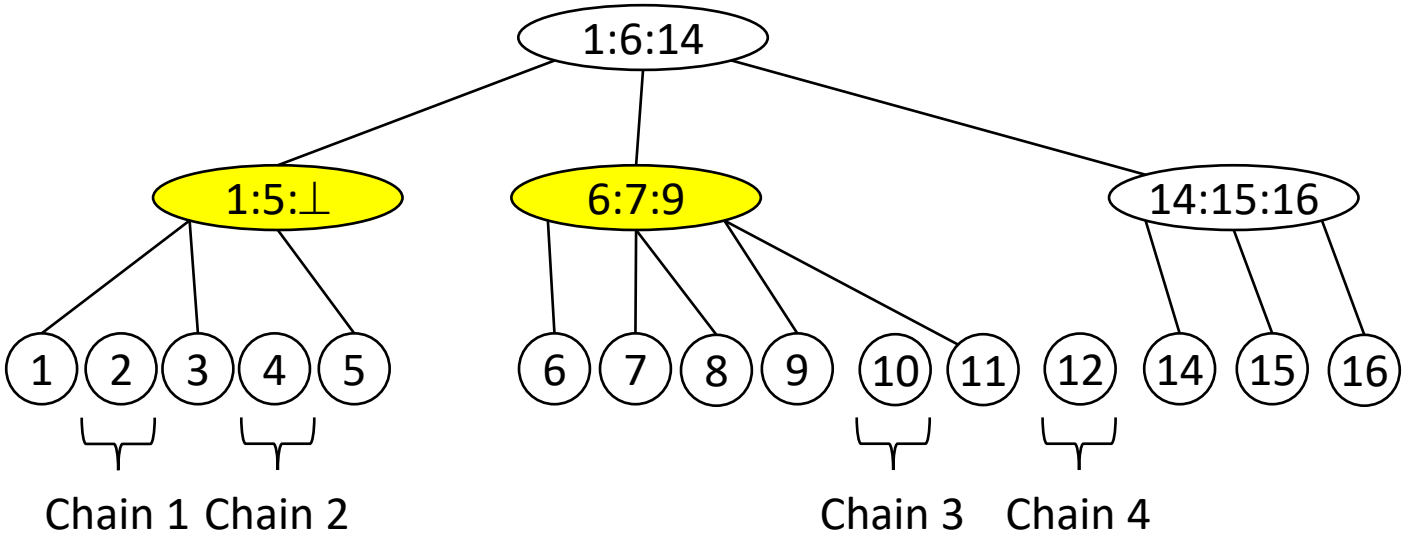
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



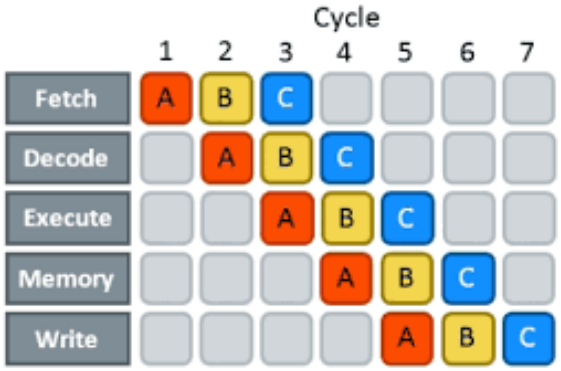
Insert elements

2 3 4 8 10 11 12



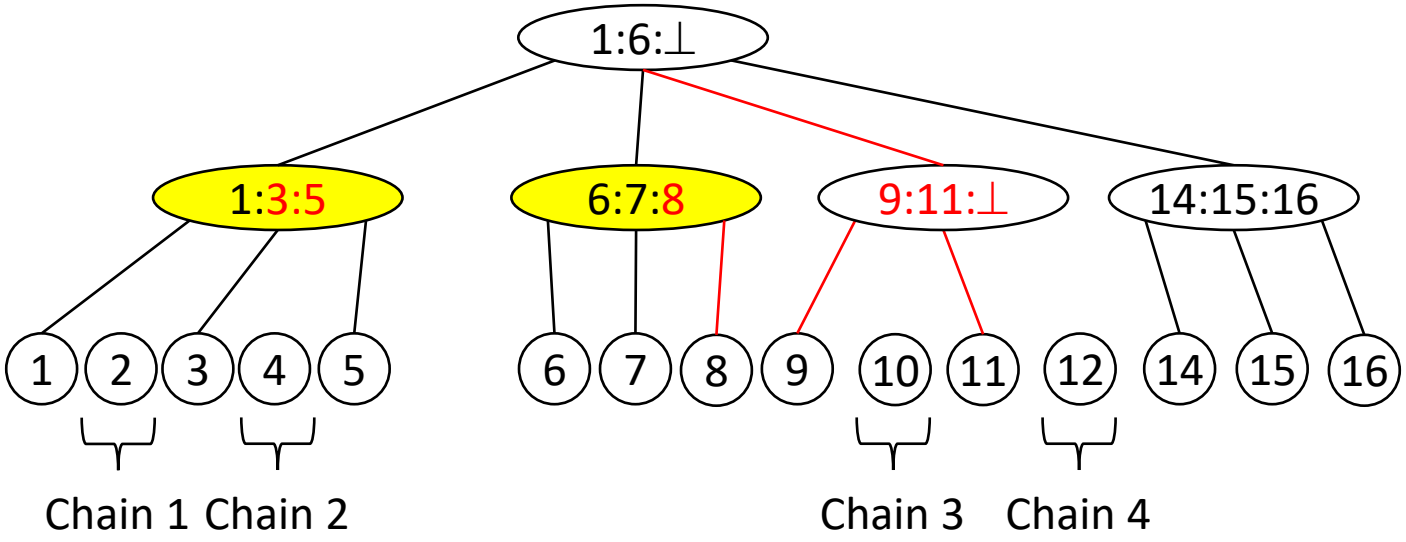
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



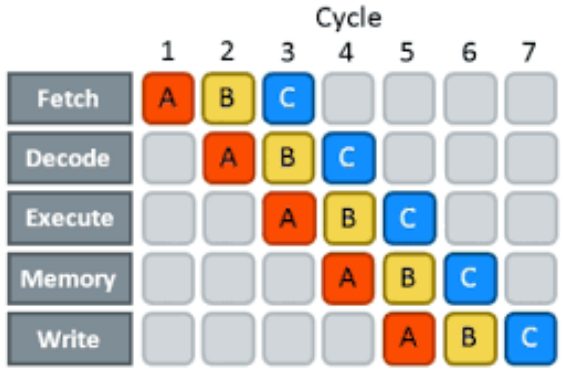
Insert elements

2 3 4 8 10 11 12



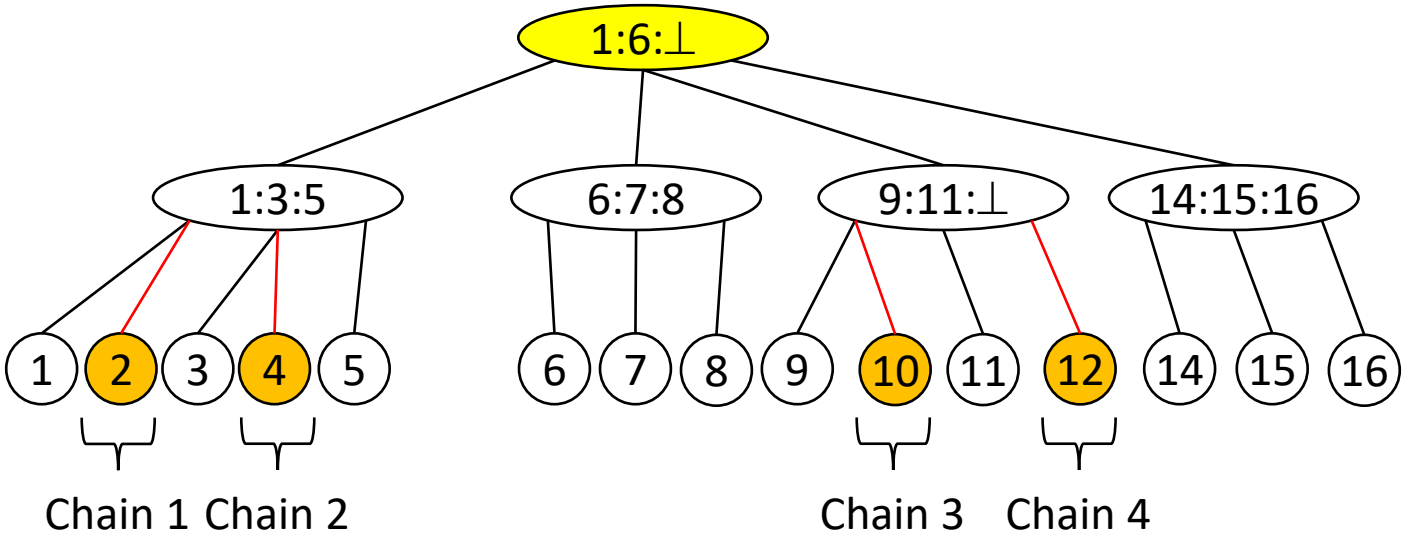
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



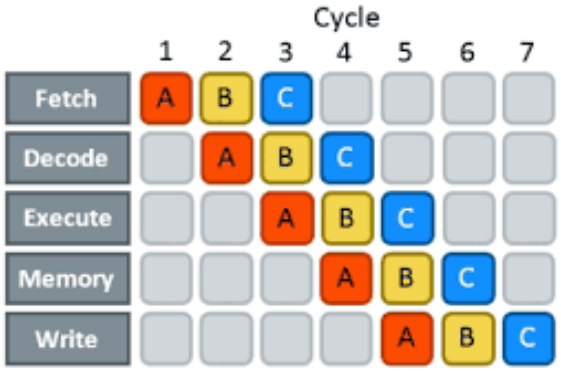
Insert elements

2 3 4 8 10 11 12



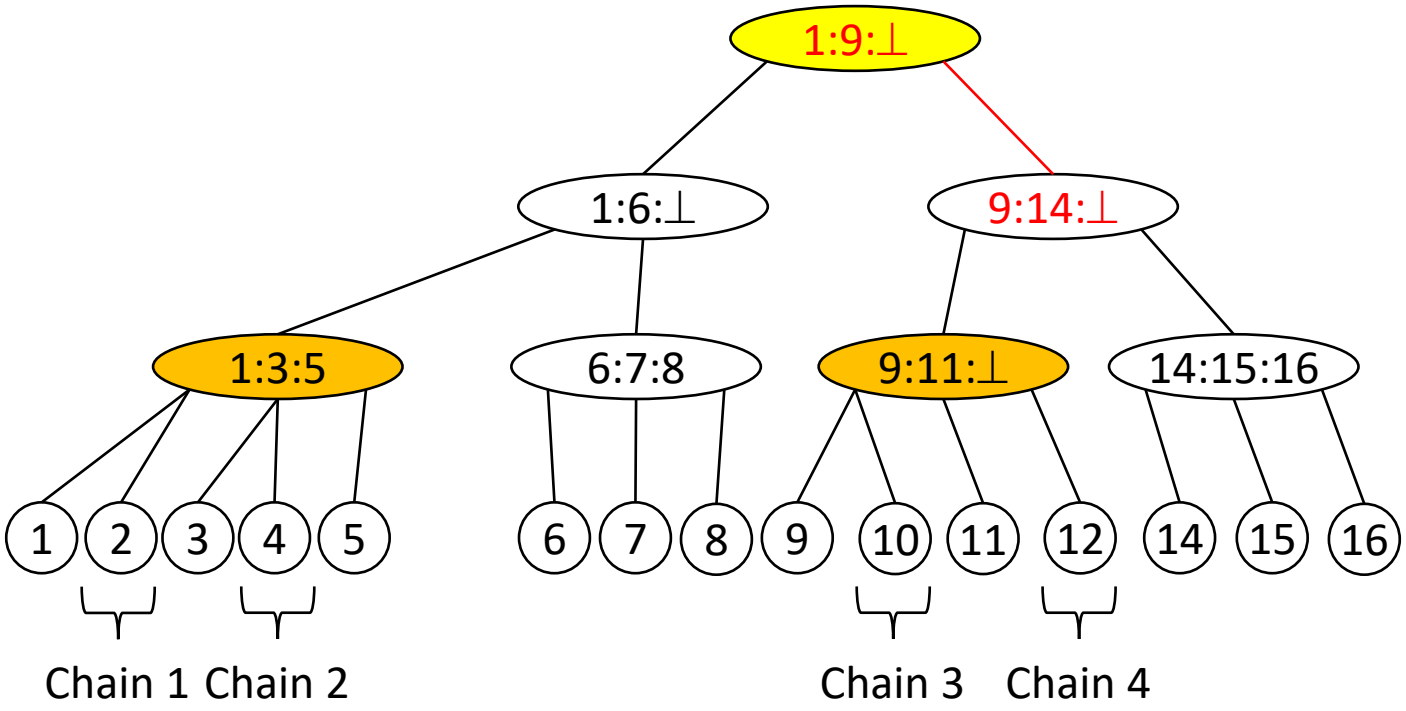
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



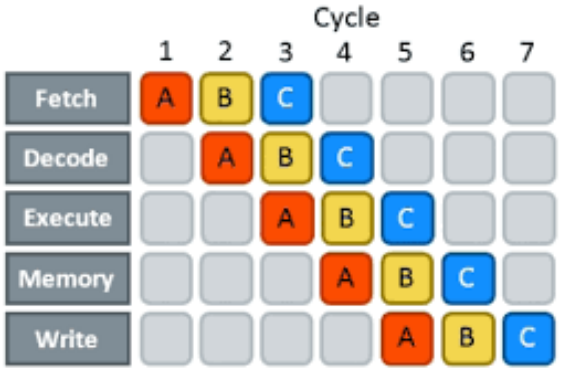
Insert elements

2 3 4 8 10 11 12



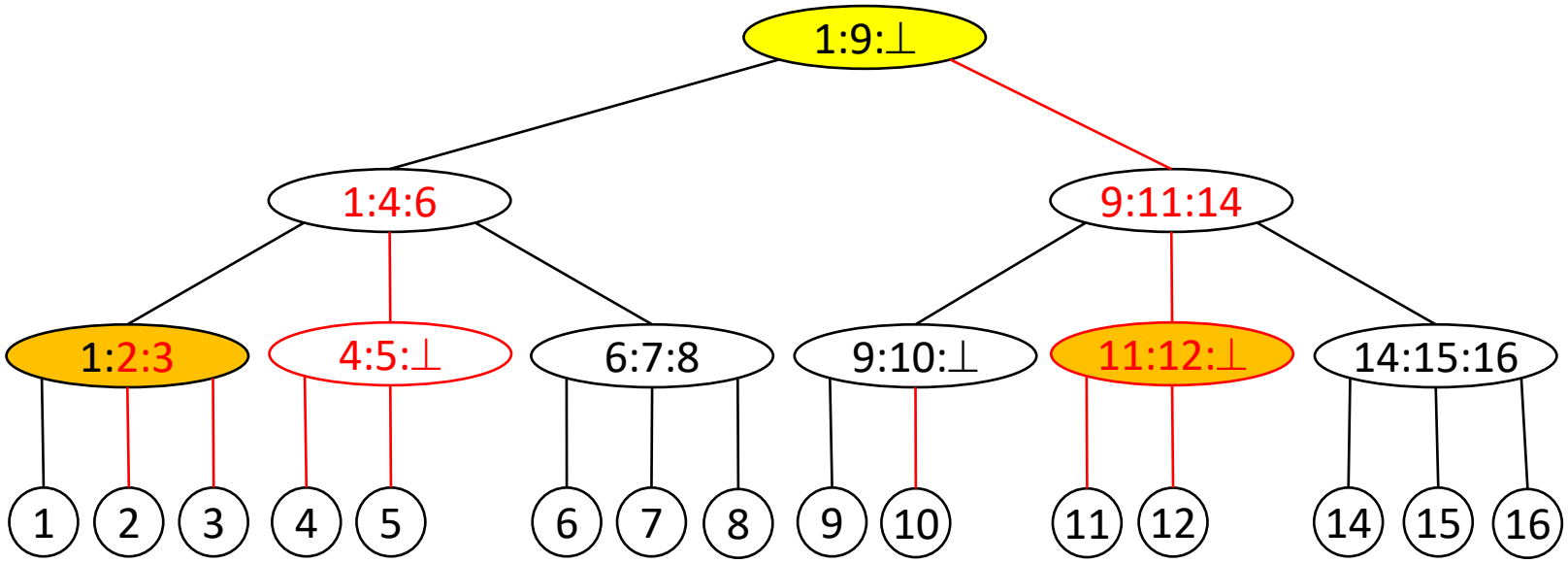
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



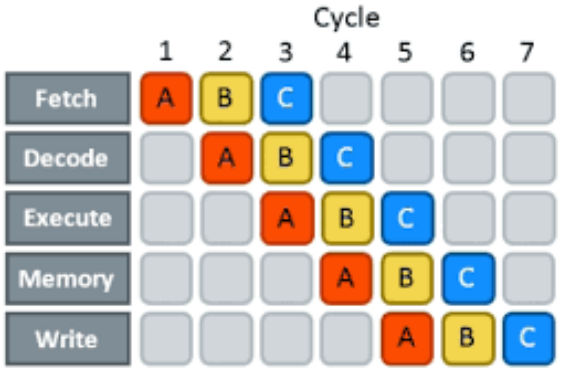
Insert elements

2 3 4 8 10 11 12



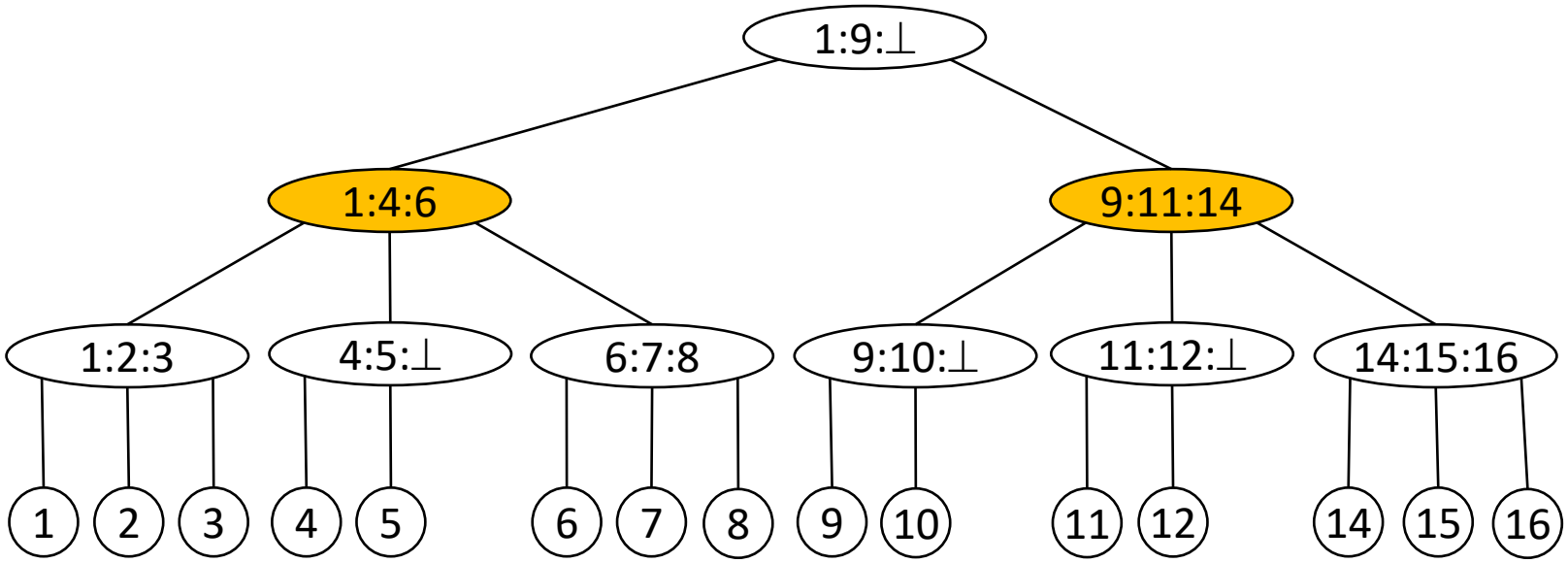
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



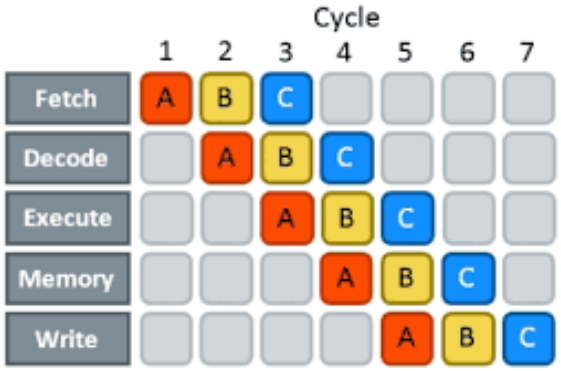
Insert elements

2 3 4 8 10 11 12



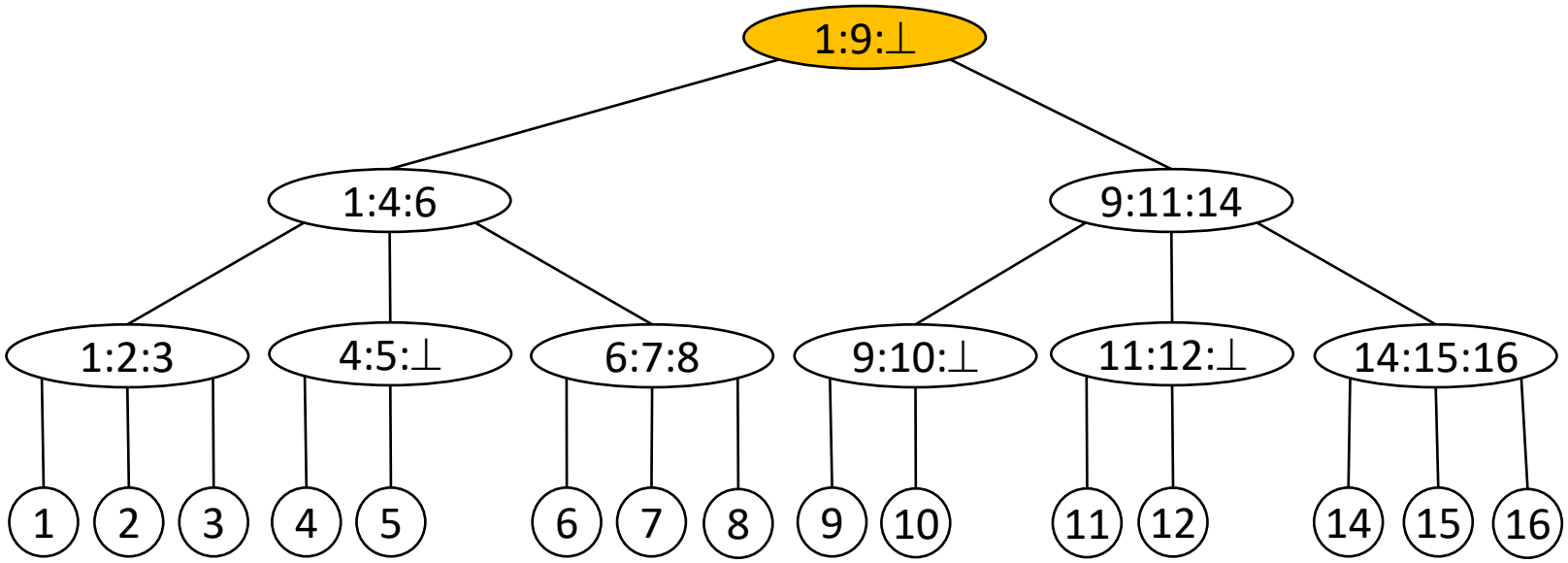
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



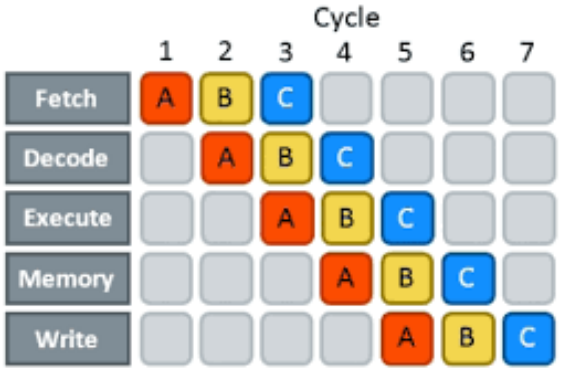
Insert elements

2 3 4 8 10 11 12



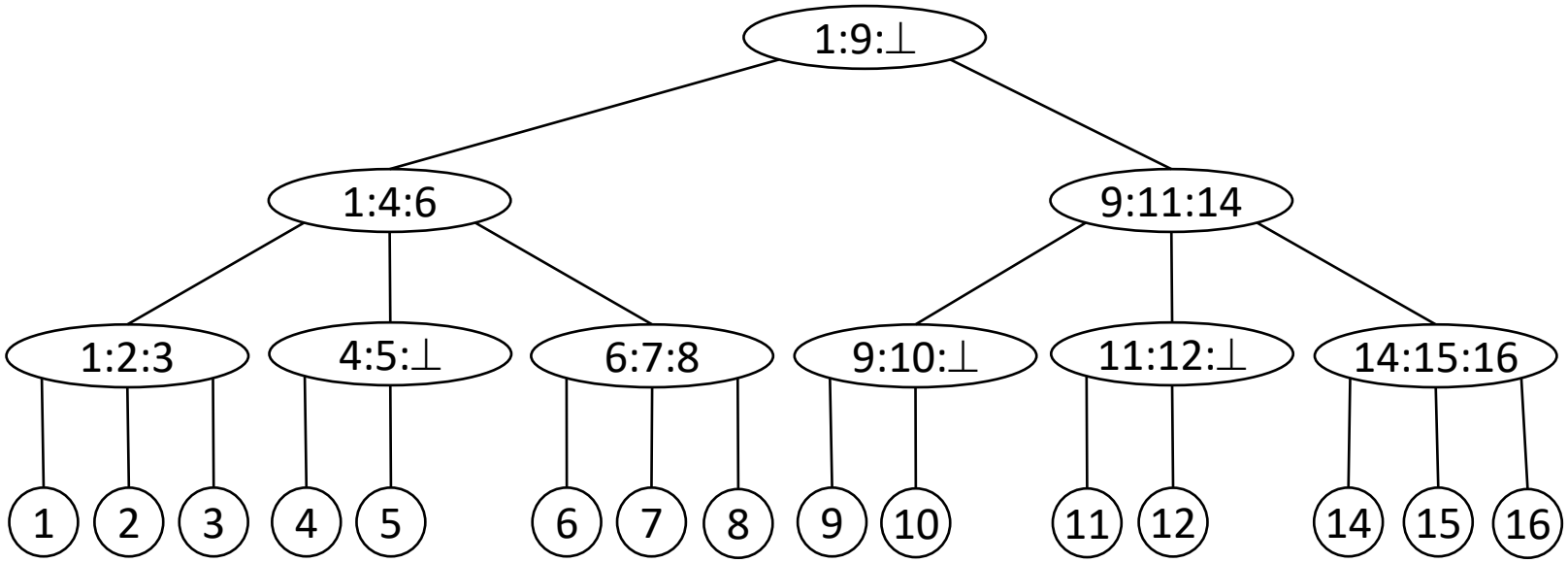
Pipelining: Inserting k elements in a (2,3)-tree

- Once the concurrent process of adding the median elements of the main chains has moved two levels up on the tree, recurse on the sub-chains



Insert elements

2 3 4 8 10 11 12



Time $O(\log k + \log n) = O(\log n)$ and work $O(k \log n)$ on a CREW PRAM

Pipelining: Inserting k elements in a (2,3)-tree

THEOREM: Given a (2,3)-tree data structure with n leaves and $k = O(n)$ new pre-sorted elements, the problem of inserting the new elements in the (2,3)-tree can be solved in time $O(\log n)$ and work $O(k \log n)$ on a CREW PRAM.

Check your understanding

- Prove the following theorem

THEOREM: Given a (2,3)-tree data structure with n leaves and k new elements, with $k = O(n)$, the problem of inserting the k new elements in the (2,3)-tree can be solved in time $O(\log n)$ and work $O(k \log n)$ on an <u>EREW PRAM</u>.

HINTS:

- Where do we have concurrent reads on same shared variables? How can we avoid them?

Check your understanding

- Develop an EREW PRAM algorithm that, given a sorted array A of n elements, constructs a (2,3)-tree that represents A in time $O(\log n)$ and work $O(n)$.
- Given a (2,3)-tree data structure with n leaves and k new pre-sorted elements, with $k = \Omega(n)$, develop an EREW PRAM algorithm that inserts the new elements in the (2,3)-tree in $O(\log k)$ time and $O(k)$ work.